Practical Work Sheet No. 1

[Machine Learning]

Linear Regression and Logistic Regression

Objectives

The objective of this practical work is to implement a complete Machine Learning workflow in Python, from data exploration to model evaluation

1. Set up a Python development environment for Machine Learning.
2. Understand how linear and logistic regression models work.
3. Implement linear regression with the scikit-learn library, then logistic regression *from scratch* (sigmoid function, cost function, gradient descent).
4. Study the impact of L1 (Lasso), L2 (Ridge) and Elastic Net (L1 + L2) regularization on learning.
5. Evaluate and interpret model performance using appropriate metrics.

Description of the work

This lab is divided into two parts:

- The first is devoted to **linear regression** on a real dataset: predicting salary from professional experience.
- The second deals with **logistic regression** applied to a binary classification problem: predicting the presence of heart disease from clinical features.

You will work in **Python**, using **scikit-learn**, together with **pandas**, **NumPy** and **Matplotlib** for data manipulation and visualization. The classification part will be implemented **entirely from scratch**, without resorting to scikit-learn estimators, in order to consolidate the understanding of the internal mechanisms of supervised learning.

1. Setting Up the Environment

Perform the following steps once, then keep this configuration for the following labs.

- **Install and check the development environment:** Python ≥ 3.10, Jupyter Notebook, Libraries: `numpy`, `pandas`, `matplotlib`, `seaborn`, `scikit-learn`

From a command prompt, run:

```
pip install numpy pandas matplotlib seaborn scikit-learn jupyter
```

Note: students without a local installation can use Google Colab, which includes all of these libraries.

a. **Create a notebook** named TP1_Regression.ipynb.

b. **Import the required libraries and check that they work properly** by running the following cell:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn

print("numpy        :", np.__version__)
print("pandas       :", pd.__version__)
print("scikit-learn :", sklearn.__version__)
```

No error should occur. If an import fails, reinstall the corresponding library (`pip install <library_name>`).

Once the environment is validated, the first experimental part can begin.

2. Linear Regression: Predicting Salary from Experience

The goal is to build a simple linear regression model to estimate salary as a function of experience. In this lab, we will use a real dataset `Salary_Data.csv`, available on Kaggle Employee Salaries Dataset. Download the file and place it in the same directory as the notebook.

Dataset variables:

Variable	Role	Description
`YearsExperience`	Explanatory (X)	Years of professional experience
`Salary`	Target (y)	Annual salary

You are asked to:

a. Load the dataset with pandas (`pd.read_csv`), then display the first rows (`df.head()`), the column types (`df.info()`) and the descriptive statistics (`df.describe()`).

b. Visualize the relationship between variables: plot the Salary vs Experience scatter plot (Matplotlib or seaborn) and compute the Pearson correlation coefficient (`df.corr()`). Comment on the linearity of the relationship.

c. Split the data into training (70 %) and test (30 %) sets with `train_test_split` (`random_state=42`). **Standardize the variables if necessary**.

d. Train a linear regression model (`sklearn.linear_model.LinearRegression`) on the training set, then display the estimated parameters and interpret the slope (`coef`, `intercept`) in business units.

e. Predict the values on the test set.

f. Visualize the results: overlay the regression line on the scatter plots of the training and test sets.

g. Compute and display the mean squared error (MSE) and the coefficient of determination **R^2** (`mean_squared_error`, `r2_score`) on the test set.

h. Add L1, L2 and Elastic Net (L1 + L2) regularization, and compare the performances: train `Ridge` (L2), `Lasso` (L1) and `ElasticNet` for several values of the `alpha` parameter (for example 0.1, 1, 10), then record MSE and R^2 in a summary table.

Table to complete:

Model	alpha	MSE (test)	R^2 (test)	Estimated coefficient
LinearRegression	/			
Ridge (L2)	0.1 / 1 / 10			
Lasso (L1)	0.1 / 1 / 10			
ElasticNet	0.1 / 1 / 10			

3. Logistic Regression: Predicting Heart Disease

The aim of this part is to build a logistic regression model to predict the presence of heart disease from several clinical features. The dataset used is the well-known Heart Disease Dataset (Cleveland Clinic Foundation), available in the UCI Machine Learning Repository; it will be loaded directly through the OpenML interface of scikit-learn (`fetch_openml(name='heart-disease', version=1)`).

Unlike Part 2, **no scikit-learn estimator will be used for learning**: the model (sigmoid function, cost function, gradient descent, prediction) will be implemented entirely with NumPy.

a. Loading and Preparing the Data

- **Load the dataset** via `fetch_openml(name='heart-disease', version=1, as_frame=True)` and display the first five rows. Check for missing values (`df.isnull().sum()`) and handle them if necessary.
- **Prepare the data**: separate the `target` (converted to integer) from the features, normalize with `MinMaxScaler` (scaling to [0, 1] — essential for the stability of gradient descent), then split into training (70 %) and test (30 %) sets, `random_state=42`.

b. Implementing the Model

Build the model step by step, in the following order:

- **sigmoid(z)**: return 1 / (1 + np.exp(−z)).
- **initialize_parameters(n):** initialize ***w*** as a zero vector of shape (n, 1) and b = 0.
- **compute_cost_and_gradients(X, y, w, b, lambda_=0.0):** compute the cross-entropy cost and the gradients ***dw***, ***db***. The `lambda_` parameter is ignored in this base version (no regularization).
- **gradient_descent(X, y, w, b, alpha=0.1, num_iters=1000, compute_fn=None, lambda_=0.0):** training loop: at each iteration, compute cost and gradients through `compute_fn`, update the parameters, store the cost. Return ***w***, ***b*** and the list of costs.
- **predict(X, w, b):** compute the probabilities ***σ(Xw + b)*** then return class 1 if probability ≥ 0.5, otherwise 0.

c. Training and Evaluation (Without Regularization)

- **Train the model** with α = 0.1 and 1000 iterations, starting from parameters initialized to zero.
- **Visualize the convergence**: plot the evolution of the cost over the iterations; the cost must decrease monotonically. If the cost diverges or stagnates, check the sigmoid function and the sign of the updates.
- **Evaluate the accuracy** of the model on the test set (`accuracy_score`) and **display the confusion matrix** (`confusion_matrix`). Interpret the four cells (true positives, true negatives, false positives, false negatives) in the medical context: which type of error is the most costly?

4. Regularization (L2, L1, Elastic Net)

You are asked to:

- **Study L2 regularization**: the code of `compute_cost_and_gradients_L2(X, y, w, b, lambda_)` is provided complete in the notebook. Analyze it line by line (penalty term in the cost, additional term in dw, absence of regularization of db).
- **Train the model with L2**: α = 0.5, 2000 iterations, λ = 1.0. Plot the cost convergence, then evaluate accuracy and confusion matrix on the test set.
- **Implement L1 regularization**: complete the function `compute_cost_and_gradients_L1(X, y, w, b, lambda_)` (*TODO cell* — hint: `np.sign(w)` for the gradient). Train with α = 0.5, 2000 iterations, λ = 0.1, then evaluate as in b.
- **Implement Elastic Net**: write `compute_cost_and_gradients_EN(X, y, w, b, lambda_, rho=0.5)` by combining the two penalties above. Train and evaluate.

5. Analysis Questions

Answer concisely:

1. Why is normalization (`MinMaxScaler`) essential before gradient descent, whereas it is optional for the linear regression?

2. Compare the convergence curves with and without regularization: does regularization slow down or stabilize the learning?

3. On the Heart Disease Dataset, which type of error (false positive vs false negative) is the most critical from a medical point of view? Which decision threshold (0.5 by default) would you recommend?

4. What does the comparison of the weights on the clinical features reveal? Do the variables selected by L1 correspond to the known risk factors?

5. Synthesis: in which scenario should Ridge, Lasso or Elastic Net be preferred?